CPT102 Data Structures: List-based Data Structures

Thomas Selig

XJTLU

March 9, 2026

Contents of today's lecture

All about the list-based (linear) data structures.

- Introducing the List abstract data type (ADT).
- The most common linear structures: ArrayList and LinkedList.
- Stacks and queues re-visited.
- Throughout, we look at:
  - Standard implementations;
  - Efficiency of operations.

We will present some of this using Java, but it can all be done in any language.

Abstract data types: what?

Definition

An **abstract data type** (ADT)is a specification of a data type, independent of an implementation. It consists of a type name and a set of specified operations.

- *Specified* operations ⇒ We know *what* the operations do: their pre- and post-conditions, and the effect of the operation.
  - We can think of the specification of an operation as its *contract*.
- *Abstract* ⇒ Implementation details are not given: we don't know *how* the data is stored, or *how* the operations are implemented.
- Example: Queue (Week 1)
  - Type is named *Queue*;
  - Specified operations: `enqueue(element)`, `dequeue()`, `peek()`, `size()`.

Abstract data types: why?

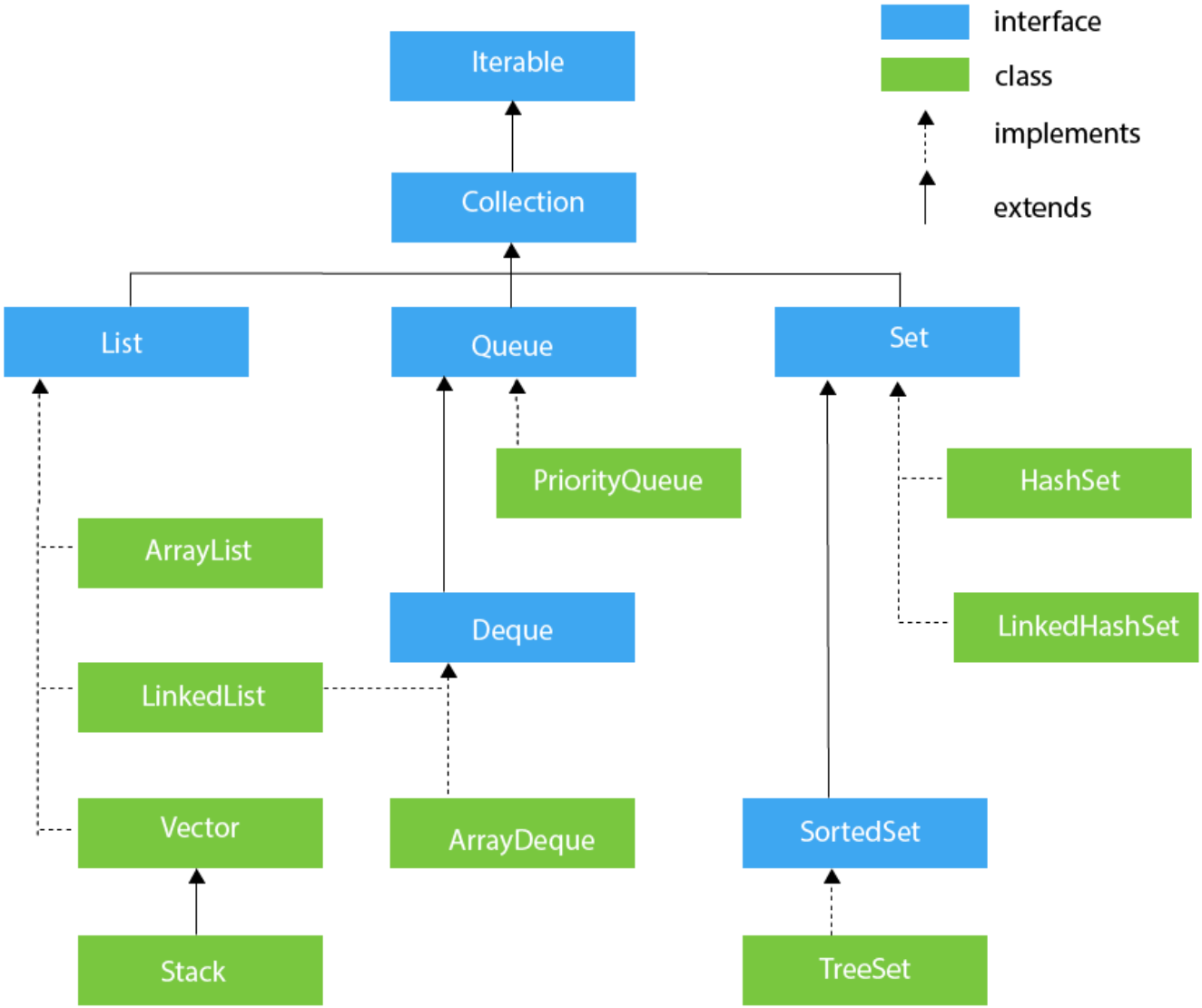
- ADTs allow the programmer to think more *generally* and *conceptually*:
  - Start with most general properties required for our data structure without worrying about implementation details.
  - Reduces cognitive load.
- Language-independence: ADTs can be defined independently of any programming language.
- Within a programming language, using ADTs provide flexibility: can substitute for any concrete implementation without breaking the code.
  - Supports concepts/principles such as *polymorphism*, *dependency inversion principle*, or "program to an interface" (see Tutorial).

Reminder: Java collection framework

- From the Java tutorial here: "a unified architecture for representing and manipulating collections".
- It contains:
  - interfaces: **ADTs** that represent different sorts of containers; `List`, `Set`, `Map`, etc.;
  - implementations: concrete **realisations** of the ADTs; `ArrayList`, `HashSet`, `HashMap`, etc.;
  - algorithms: operations (methods) that manipulate the collections and perform useful operations on them (search, sort, add, etc.); the operation is specified on the ADT, and its implementation depends on the concrete realisation.
- The collection framework is divided into two categories.
  1. `Collection` objects for collections of values only.
  2. `Map` objects for collections of pairs (Key, Value).

  For most of the course, we will only look at `Collection`.

The Collection hierarchy



The `Collection` interface

- The root interface in the collection hierarchy.
- A collection is a group of objects, known as *elements* in the collection.
- Different implementations: with/without duplicates, ordered/unordered, etc..
- `Collection` operations: `add()`, `remove()`, `contains()`, `size()`, `isEmpty()`, and a few others; see here.
  - Most general operations for a collection of objects. Any collection of objects should have these.
  - But how they are *implemented* depends on the type of collection.
- `Collection` extends `Iterable`, which means we can iterate over any (implementation of) `Collection` (iterate = inspect each element in the Collection one-by-one).
  - In this sense, a Bag (Week 1) is not a collection.
  - Should we consider stacks and queues to be collections?

The three sub-types of Collection

The Collection interface has no direct implementation: instead classes implement more specific subinterfaces.

1. List: an ordered collection of elements, with possible duplicates.
2. Queue: a collection for holding elements prior to processing; generally first-in first-out (FIFO).
3. Set: an unordered collection of elements, without duplicates.

Today, we will focus on List.

The List ADT

Definition

A **list** is a linear, ordered ADT. It consists of a finite sequence of data items known as *elements*, ordered by *index* or *position*.

Additional constraints on Collection operations:

- Addition: `add(element)` adds to the *tail* of the list (highest index);
- Deletion: `remove(element)` removes the first (by index) occurrence of `element`.
- Iteration: order follows indexing, from head to tail.

List-specific operations:

- Positional access: `get(index)`, `set(index, element)`;
- Positional search: `indexOf(element)`, `indexOf(startIndex, element)` `lastIndexOf(element)`;
- Positional modification: `insert(index, element)`, `remove(index)`;
- Range-view: `sublist(startIndex, endIndex)`.

Data structure model

In this course, we will consider:

- ADTs: conceptual specification = name + specified operations;
- Data structures: concrete implementations/realisations of ADTs. A data structure includes:
  - Data model = how the structure stores the data;
  - Algorithms = implementations of the specified operations.

Remark

This model of data structures closely mimics that of a class in Object Oriented Programming (OOP): data + methods/algorithms.

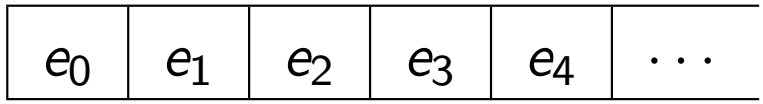
List-based data structures

There are two common realisations of the List ADT.

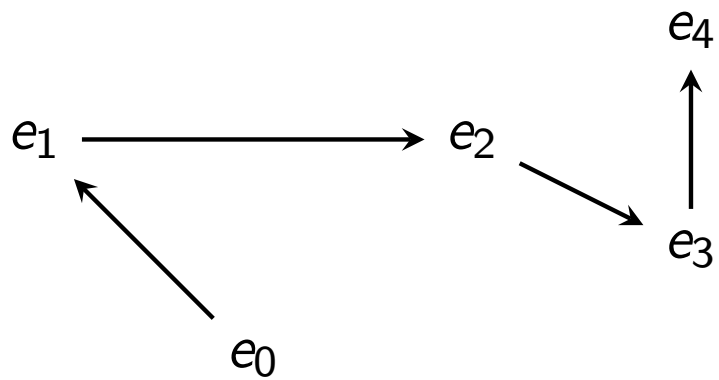
1. ArrayList: resizeable array implementation
   - Data model is an array with additional space for new elements.
2. LinkedList: stores elements through dynamic allocation of link nodes.
   - Has multiple variants: singly-linked, doubly-linked, circular, etc.
   - For now we will consider singly-linked lists: each element points to its successor in the list.

ArrayList vs LinkedList

- ArrayList is implemented as an array: contiguous block of memory, with extra capacity at the end (for new elements).



  - Fast access (retrieval) and iteration.
  - When inserting/deleting, need to shift elements.
- LinkedList elements can be anywhere in memory. Each element is stored inside a *node* which also holds a *pointer* to its *successor* in the list.



  - Can be faster for insertion and deletion.
  - Access needs to traverse the list.

ArrayList vs LinkedList complexity: `get()`

- List with n elements, what is the complexity of accessing an element, i.e. of `get(k)`?
  - Average-case: k is a random integer in $[0, n)$.
  - Worst-case: take the max over $k \in [0, n)$.
- Analyse using Big-Oh notation (Week 2).
- Will then look at other operations: `add()`, `remove()`, `contains()`.
- Usually same for worst- and average-cases.

ArrayList complexity: `get(k)`

For ArrayList (or array[k]): variable just stores reference to memory address of initial element.

- First, access initial element at head of ArrayList.

e_0	e_1	e_2	e_3	e_4	$\cdots$

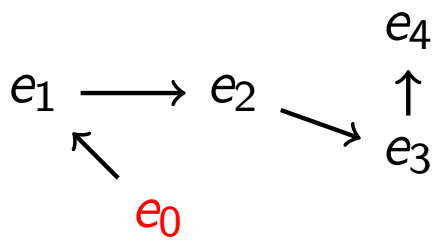
  Retrieving a reference = constant $\mathrm{O}(1)$ time.
- Then, because arrays are contiguous in memory, just need to *offset* by k (see Week 1).

e_0	e_1	e_2	e_3	e_4	$\cdots$

  Equivalent of an operation "+= k", so constant $\mathrm{O}(1)$ time.
- Total complexity: $\mathrm{O}(1)$.

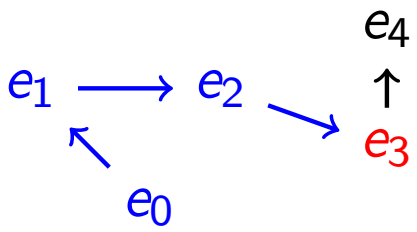
LinkedList complexity: `get(k)`

- First, access initial element at head of LinkedList.



  Retrieving a reference = constant $\mathrm{O}(1)$ time.

- Then, need to *traverse* the list until we get to k-th element, i.e. visit elements one-by-one.



  - Complexity = $\mathrm{O}(k)$

- Average- and worst-case complexities are $\mathrm{O}(n)$ (linear).

Towards more rigourous models and analysis

- Previous slides are a little "hand-wavy".
- In order to be more rigourous, we need a more precise model for our data structure, in particular how the elements are stored.
- However, we still want our models to remain language-independent.

Our choice is to use the *class* model:

- Data structures as classes (in an OOP sense) with instance variables and algorithms/procedures (methods, in OOP terminology);
- We will write `Struct.var` to refer to the instance variable `var` in the structure `Struct`.
- When writing a method in pseudocode, we will write the class instance as the first parameter of the procedure.

Array-based data structure model

For data structures which are *array-based*, our model is:

```
ArrayStructure<E>:
    E[] elements;
    int size;
```

- `E` is the type of the elements in our data structure, meaning our data structure is *generic*.
- The array `elements` satisfies that exactly its first `size` elements are non-null.
- We also assume that there is a CheckCapacity() procedure operating on our structure that ensures there is always sufficient space remaining in the array.
  - If not, the operation copies all the elements into a new, larger array through a Resize() procedure.
  - Resize() has time and space complexity $\mathrm{O}(n)$, but is rarely called.

Singly-linked list model

For singly-linked lists, our model is:

```
SinglyLinkedList<E>:
    Node<E> head;
    Node<E> tail;
    int size;
Node<E>:
    E data;
    Node<E> next;
```

- A node stores an element and the next node in the list.
- The linked list stores the head and tail nodes.
- We must have `head.next.next.`$\cdots$ (`size` - 1 times) $=$ `tail`, and `tail.next` $=$ `null`.

Pseudocode for get()

For ArrayList:

Procedure GET(`list`, k)
return `list.elements`$[k]$;

For SinglyLinkedList:

Procedure GET(`list`, k)
$i \leftarrow 0$, `current` $\leftarrow$ `list.head`;
while $i < k$ **do**
  `current` $\leftarrow$ `current.next`;
return `current.data`;

- When implementing in code, you should also do a bounds check $0 \leq k <$ `list.size`.
- Complexity analysis is straightforward, we get:
  - Time complexity: $\mathrm{O}(1)$ for ArrayList, $\mathrm{O}(n)$ for SinglyLinkedList;
  - Space complexity: $\mathrm{O}(1)$ for both.

ArrayList complexity: `add(element)`

- Recall that for list-based structures, `add()` adds (appends) the new element at the tail of the list.
- First, access empty space at tail of ArrayList.

e_0	e_1	e_2	e_3	e_4	$\emptyset$	$\cdots$

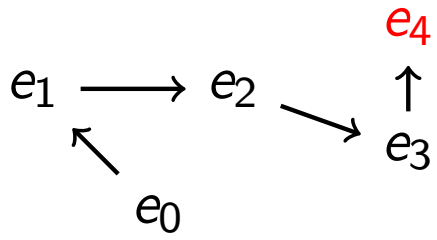
  Like `get()`, constant time $\mathrm{O}(1)$.
- Then, just need to put new element in first empty space.

e_0	e_1	e_2	e_3	e_4	e_5	$\cdots$

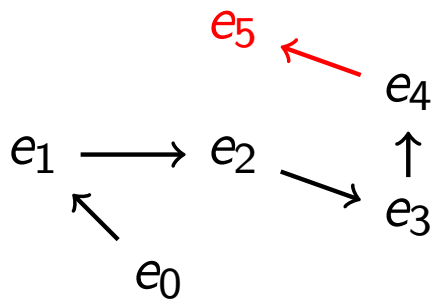
  Equivalent of a "set" operation (array[k] = e), so $\mathrm{O}(1)$.
- Total complexity: $\mathrm{O}(1)$.
- BUT!!! If no space left, i.e. have reached *capacity*, need to copy all the array into a new space using RESIZE(), so $\mathrm{O}(n)$.

LinkedList complexity: `add(element)`

- First, access initial element at tail of LinkedList.



  Because LinkedList stores references to head and tail, this is $\mathrm{O}(1)$.
- Then, just create new links to new element e_5.



  Creating a link is just assigning a variable, so $\mathrm{O}(1)$.
- Total complexity: $\mathrm{O}(1)$.

Pseudocode: add(element)

For ArrayList:

Procedure ADD(list, element)
CHECKCAPACITY(list); ▷ possible resizing required
list.elements[size] ← element;
list.size ← list.size + 1;

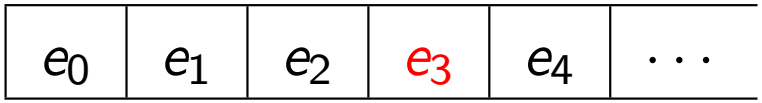
For LinkedList:

Procedure ADD(list, element)
new_tail_node(data=element, next=null);
▷ create a new node, which will be the new tail of our list
list.tail.next ← new_tail_node;
▷ old tail node points to our new tail node
list.tail ← new_tail_node; ▷ set the tail to our new tail node
list.size ← list.size + 1;

Here, we assume the LinkedList is non-empty.

ArrayList complexity: `insert(index, element)`

- First, access k-th element of ArrayList.



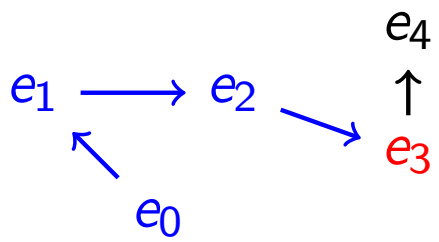
  Like `get()`, constant time $\mathrm{O}(1)$.
- Then, need to insert new element, which means shifting all remaining elements by one place.

  Complexity = number of elements to shift = $\mathrm{O}(n-k)$.
- Average and worst cases are both $\mathrm{O}(n)$.

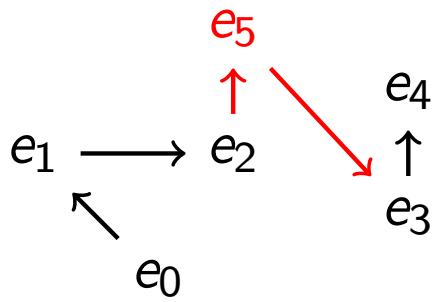
Procedure INSERT(`list`, `index`, `element`)
CHECKCAPACITY(`list`);
$k \leftarrow$`list.size`;
while $k >$ *index* **do**
  `list.elements`$[k] \leftarrow$ `list.elements`$[k-1]$;
`list.elements[index]` $\leftarrow$ `element`;
`list.size` $\leftarrow$ `list.size` $+ 1$;

LinkedList complexity: `insert(index, element)`

- First, access k-th element of LinkedList.



  Like `get(k)`, so $\mathrm{O}(n)$.
- Then, just break old links $e_2 \leftrightarrow e_3$, and create new links to e_5 instead.



  All this is constant time $\mathrm{O}(1)$.
- Total complexity is therefore $\mathrm{O}(n)$.
  - BUT much faster if k $\ll$ n.

LinkedList `insert()`: pseudocode

```
Procedure INSERT(list, index, element)
if index = 0 then
    new_head_node(data=element, next=list.head);
    list.head ← new_head_node;
else
    k ← 0, previous ← list.head;
    while k < index - 1 do
        previous ← previous.next;
    ▷ traverse to previous = list.get(index - 1)
    new_node(data=element, next=previous.next);
    previous.next ← new_node;
list.size ← list.size + 1;
```

As previously, we assume the list is non-empty here.

Comparison of `insert()`: ArrayList vs LinkedList

- For both structures, the insertion operation generally has linear complexity.
- Cost is borne by different parts of the algorithm:
  - For ArrayList, cost comes from shifting array after insertion index.
  - For LinkedList, cost comes from traversing to the insertion index.
- LinkedList has fast insertion near the head of the list.
- ArrayList has fast insertion near the tail of the list (providing no resizing).
  - Also the case for doubly-linked lists, which therefore have fast insertion near both head and tail of the list.
- In general, LinkedList is less efficient in memory resources, since needs to maintain pointers through the list.

Demo!

Activity: ArrayList vs LinkedList complexities

Complete the table below, using similar arguments to the first three methods studied.

Method	ArrayList	LinkedList
`get(index)`	O(1)	O(n)
`add(element)`	usually O(1), but (very) rarely O(n)	always O(1)
`insert(index, element)`	O(n)	O(n)
`remove(element)`	??	??
`remove(index)`	??	??
`contains(element)`	??	??

Test your answers in the LMO Quiz here.

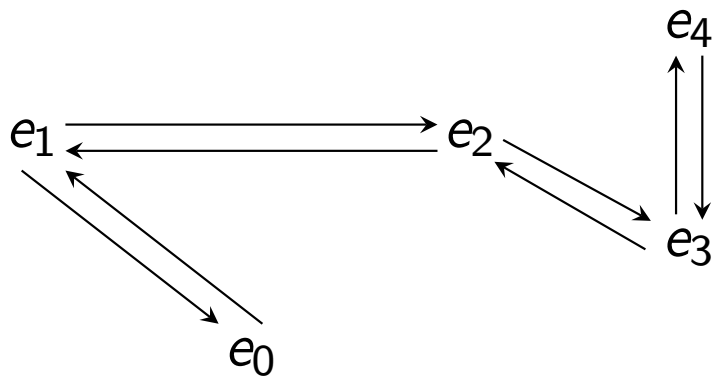
ArrayList vs LinkedList complexities: summary

Method	ArrayList	LinkedList
`get(index)`	O(1)	O(n)
`add(element)`	usually O(1) (very) rarely O(n)	always O(1)
`add(index, element)`	O(n)	O(n)
`remove(...)`	always O(n)	O(n) for general element/index faster if element is near the head / index is small
`contains(element)`	O(n)	O(n)

Doubly-linked lists: what?

- In a doubly-linked list, each node stores pointers to both *predecessor* and *successor* in the list.
- Data structure model:

```
DoublyLinkedList<E>:
    Node<E> head;
    Node<E> tail;
    int size;
Node<E>:
    E data;
    Node<E> next;
    Node<E> previous;
```



- Java implements its `LinkedList` class as a doubly-linked list.
  - Allows for faster positional operations near the tail of the list.

Doubly-linked list example: `get()`

```
Procedure GET(list, k)
if k < list.size / 2 then
    i ← 0, current ← list.head;
    while i < k do
        current ← current.next;
else
    i ← list.size − 1, current ← list.tail;
    while i > k do
        current ← current.previous;
return current.data;
```

- Time complexity is $\mathrm{O}(m)$, where $m = \min\{k, n - k\}$.
- Similar for `insert()` operation.
- Can also have separate `addFirst()` and `addLast()` operations.

Stacks revisited: model

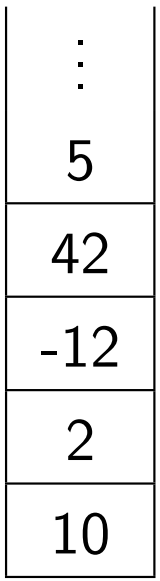
Definition

A **stack** is a linear (list-based) data structure in which elements may be inserted or removed from only one end (= the *top*).

- Data model is array-based:

```
Stack<E>:
    E[] elements;
    int size;
```

- Can also use a doubly-linked list model, but arrays are faster (hardware).
- Operations: `push(element)`, `pop()`, `peek()`, `size()`.
  - No iteration or searching!



Stacks revisited: algorithms

Procedure PUSH(stack, element)
CHECKCAPACITY(stack);
stack.elements[stack.size] ← element;
stack.size ← stack.size + 1;

Procedure PEEK(stack, element)
if *stack.size* = *0* **then**
| **FAIL**; ▷ empty stacks contain no elements
return stack.elements[stack.size];

Procedure POP(stack, element)
if *stack.size* = *0* **then**
| **FAIL**; ▷ cannot pop an empty stack
stack.size ← stack.size - 1;
temp ← stack.elements[stack.size];
stack.elements[stack.size] ← null;
return temp;

- In fact, POP() can also check the capacity to avoid excessive waste.
- All complexities are $\mathrm{O}(1)$ (constant) except when needing to resize.

Queues revisited

Definition

A **queue** is a linear data structure in which elements are inserted only at one end (the *tail*), and removed only from the other end (the *head*).

Two standard data models:

Array-based;

```
Queue<E>:
    E[] elements;
    int head;
    int tail;
```

LinkedList-based.

```
Queue<E>:
    Node<E> head;
    Node<E> tail;
    int size;

Node<E>:
    E data;
    Node<E> next;
```

Operations:

`enqueue(element)`, `dequeue()`, `peek()`, `size()`.

- Always constant time/space, except when resizing.
- See Exercise sheet for details.

Summary

- ADTs for data structure and operation specification.
- The List ADT and its common implementations: ArrayList and LinkedList.
- Revisiting the Stack and Queue ADTs from Week 1.
- Pseudocode to implement standard operations; complexity analysis.